

Claims Investigation Copilot — Judge Questions & Answers

Most Probable to Least Probable

#1: "Your data is 100% synthetic. How do you know this works on real claims?"

ANSWER: "Great question. We used actual OIG case studies and CMS fraud bulletins to design our fraud patterns. The synthetic data mimics real-world distributions - 95% normal claims, realistic CPT/ICD code patterns by specialty, and actual billing amounts from CMS fee schedules.

More importantly - **fraud patterns are well-documented**. Upcoding rings, phantom billing, doctor shopping - these aren't novel fraud types. The OIG publishes case details publicly. We reverse-engineered those patterns.

The synthetic data lets us **control ground truth** - we know exactly which claims are fraud, which means we can actually measure precision and recall. With real data, you often don't know what fraud you're missing.

That said - **absolutely**, the next step is pilot validation with an actual payer's anonymized data. But for proving the agent workflow and detection logic, synthetic data with known fraud patterns is actually the right starting point."

#2: "This demo feels too perfect. Is everything pre-generated?"

ANSWER: "The LLM calls are 100% live - you can see the latency. But yes, the data is cached so we don't regenerate 50,000 claims every time we demo.

Here's what's real-time:

- All 3 agents making decisions
- Tool calls to anomaly detection, network analysis, billing rules search
- The agent reasoning and evidence assessment
- The rejection loop where the Case Writer sends the Detective back for more data

What's cached:

- The 50K claims dataset
- Provider profiles and member data
- The network graph structure

Want me to run it on a different case right now? We have 4 pre-loaded fraud scenarios - upcoding ring, phantom billing, doctor shopping, and a legitimate high-biller that should be dismissed. [BE READY TO DO THIS]

The 90-second runtime is real with cached data. Cold start with data generation would be about 2 minutes total."

#3: "Why do you need 3 agents? Couldn't this just be a single prompt with tools?"

ANSWER: "We tried that first. Single agent with all 12 tools - it worked, but it was unfocused.

The separation creates **division of cognitive labor**:

- The Lead has **no tools** - it only sees results and makes strategic decisions. No tool calling = cleaner reasoning.
- The Detective has **investigation tools** - it's optimized for data gathering and pattern finding.
- The Case Writer has **compilation tools** - it's optimized for evidence assessment and dossier quality.

The key innovation is the **rejection loop**. The Case Writer can reject insufficient evidence and force the Detective to gather more data. With a single agent, we never saw that self-correction behavior - it would just compile whatever it had.

Could you do this with one agent? Probably. But we saw measurably better case quality with specialization. The rejection loop triggered in about 30% of cases during testing, and those cases had stronger final dossiers."

#4: "You say fraud costs \$100B annually - how much could your system realistically save?"

ANSWER: "Conservative math: A fraud analyst at a regional payer handles maybe 2-3 full investigations per day. That's 500-750 cases per analyst per year.

Our system can run an investigation in **90 seconds**. That's roughly **400 investigations per work day** per system instance if you throttle for cost.

But the real value isn't replacing analysts - it's **triage**. Most alerts die in queue because analysts can't get to them. Our system could **pre-investigate every alert** and rank them by evidence quality.

If a regional payer processes 10,000 alerts per year, and our system helps prioritize the top 100 serious cases (with an average recovery of \$500K per case), that's **\$50M in recoveries** versus maybe \$10-15M with manual triage.

The ROI isn't replacing humans - it's **scaling investigation capacity** so high-value cases don't slip through. One analyst using this system could handle the case volume of 5-10 analysts today."

#5: "What's the false positive rate of your anomaly detection?"

ANSWER: "In our synthetic dataset with known ground truth, Isolation Forest at 0.85+ threshold gives us:

- **Precision:** ~73% (73% of flags are actually fraud)
- **Recall:** ~89% (we catch 89% of fraud cases)
- **False positive rate:** ~2.5% of normal claims

But here's the key - **we don't act on the anomaly score alone**. The anomaly detection is just the first filter. The agents then:

1. Profile the provider vs peers
2. Map network connections
3. Search for billing rule violations
4. Assess evidence quality

The legitimate-high-biller case in our demo - the provider had a high anomaly score (0.88), but the agents dismissed it after investigation because the billing matched their high-complexity patient mix and specialty profile. **That's the entire point of the agent layer** - it does what an anomaly score can't."

#6: "Why hasn't this been built before? What changed that makes this possible now?"

ANSWER: "Three things converged in the last 18 months:

1. **Function-calling LLMs got reliable.** Claude 3.5 Sonnet and GPT-4 can actually use tools accurately now. Two years ago, function calling was too flaky for production workflows.
2. **Multi-agent frameworks matured.** LangGraph gave us state management, routing logic, and streaming that actually works. Before that, building agent loops was custom infrastructure hell.
3. **Healthcare fraud ML is commoditized.** Anomaly detection, network analysis, NLP on medical codes - these are solved problems. The bottleneck was never detection, it was **investigation** - the reasoning layer between 'flag' and 'case file.'

What's novel isn't the fraud detection - it's **using LLM agents as synthetic fraud analysts** who can work through a case like a human would: gather evidence, assess quality, compile a dossier, and reject weak evidence.

Systems like FICO Falcon detect fraud. We investigate it. That's the leap."

#7: "Can this scale to millions of claims per day?"

ANSWER: "We're architected for batch investigation, not real-time claim adjudication.

Typical workflow:

- Claims get processed through normal adjudication
- Anomaly detection runs nightly on new claims (fast - scikit-learn, not LLM)
- High-scoring claims go into investigation queue
- Agents work through queue at ~90 seconds per case

At 1M claims/day, maybe 25,000 get anomaly flags (2.5% FP rate). If we prioritize top 1,000 for full investigation, that's **25 hours of agent time** - easily parallelizable across multiple instances.

Cost at scale: ~\$0.15 per investigation in Claude API calls (8-12 tool calls × ~2K tokens avg). At 1,000 investigations/day, that's **\$150/day or ~\$55K/year** in LLM costs. A single fraud analyst costs \$80-120K/year.

Bottleneck isn't LLM throughput - it's whether we can parallelize effectively. LangGraph supports this with async execution. We'd need to test at scale, but architecturally it's designed for it."

#8: "What about existing solutions - FICO, SAS, Optum?"

ANSWER: "They're great at **detection** - flagging anomalies, building risk scores. FICO Falcon is the gold standard for transaction fraud.

But they stop at the flag. From there, it's:

1. Export the alert
2. Human analyst opens 6 different systems
3. Pull claims, provider history, peer comparisons
4. Spend 3 hours building the case file
5. Write a report citing regulations

We automate steps 2-5. We sit downstream of detection systems - we could even integrate with their APIs and use their anomaly scores as input.

Our competitive advantage isn't better anomaly detection - it's **investigation automation**. We're not competing with FICO. We're competing with the army of fraud analysts manually investigating FICO's alerts.

The analogy: Fraud detection systems are metal detectors. We're the team that digs up what the metal detector finds and determines if it's treasure or trash."

#9: "Can your dossier output be used as legal evidence?"

ANSWER: "Not as-is. This is an **investigative tool**, not a legal product.

The dossier output is designed to help analysts decide:

- Is this worth investigating further?
- What evidence exists?
- What regulations might be violated?

A human analyst would then:

1. Verify all findings manually
2. Pull authenticated source documents
3. Build the legal case file with proper chain of custody
4. Work with legal/compliance on formal action

That said - the system maintains full audit trails:

- Every tool call and output is logged
- Every LLM decision is saved with reasoning
- Source data references are preserved

So while the AI output itself isn't admissible, it creates a **roadmap for investigation** that a human can follow and properly document. Think of it like a private investigator's notes - not evidence itself, but a guide to finding evidence."

#10: "You showed legitimate case dismissal - but that felt forced. How often does that actually happen?"

ANSWER: "In our testing with 4 pre-built scenarios, the legitimate dismissal happens **100% of the time on that specific case** (Academic Medical Center with high-complexity specialty practice).

But fair criticism - we designed that case specifically to test if the agents could dismiss high-anomaly-score cases. It's a test case, not random sampling.

In reality, if you're feeding the system anomaly scores >0.85 , you'd expect maybe **10-20% to be legitimate** outliers (high-volume specialists, academic centers, rare procedures, etc.). The system would need to correctly identify those.

We've only tested 4 scenarios deeply. To truly know the dismissal rate, we'd need to run this on hundreds of real cases with known outcomes. That's the validation work needed for v1.

Honest answer: **The dismissal feature works, but we've only proven it on one designed test case.** More validation needed, and I acknowledge that's a weakness in our current evidence."

#11: "Your system requires AWS Bedrock. What about on-premise healthcare systems with strict data residency?"

ANSWER: "Data residency version is feasible but not built.

Current architecture:

- AWS Bedrock (Claude) in us-east-1
- Data processed in AWS with strict IAM controls
- HIPAA-compliant infrastructure setup

For on-premise requirements:

1. Swap Bedrock for self-hosted Llama 3.1 70B or similar (performance hit but doable)
2. Container-based deployment (already Dockerizable)
3. Keep all data processing local - only network analysis and tools run locally

Healthcare orgs with strict data residency (EU, healthcare systems with patient data) would need the self-hosted LLM version. **That's a go-to-market segment question** - do we start with cloud-friendly insurers or build on-premise first?

For MVP, we targeted cloud-forward payers. But the architecture doesn't lock us to Bedrock - LangGraph is LLM-agnostic."

#12: "What if the AI accuses an innocent provider? Who's liable?"

ANSWER: "Critical distinction: **The system doesn't accuse anyone. It investigates.**

Output goes to human fraud analysts, not to law enforcement or provider audits. The dossier is an internal document saying 'here's what we found, here's the evidence quality, here's what regulations might apply.'

Human-in-the-loop is mandatory:

- Analysts review all findings
- Compliance teams approve any action
- Legal signs off before any provider contact
- Providers get full due process during audit

Same liability framework as any fraud detection software - it's a tool that helps humans make decisions. The decision authority stays with humans.

We'd include disclaimers in the dossier output: 'AI-generated preliminary investigation. Requires human review and verification before action.'

If deployed irresponsibly (e.g., autopilot audits without review), that's a deployment failure, not a product failure. Same way Excel isn't liable if someone bases a bad business decision on a spreadsheet."

#13: "Your agents can get stuck in rejection loops. How many iterations before you force a conclusion?"

ANSWER: "Max 3 investigation cycles before forced termination.

Current routing logic:

1. Investigation → Dossier → Orchestrator evaluates
2. If insufficient evidence → Orchestrator sends back to Investigation
3. After 3 cycles, Orchestrator must make a conclusion (even if evidence is weak)

We log 'forced_conclusion' flag when this happens so analysts know the case had evidence quality issues.

In our testing:

- ~70% of cases conclude after first dossier compilation
- ~25% go through one rejection loop (Detective gathers more data)
- ~5% hit the second rejection
- We've never seen a case require forced termination at 3 cycles, but the safety limit exists

The rejection loop is valuable when it happens - those cases had noticeably stronger final dossiers. But you're right that infinite loops would be catastrophic. Three strikes and the agents must make a call."

#14: "If I gave you \$500K and 6 months, what would you build next?"

ANSWER: "Three priorities:

1. Pilot validation with real payer data (Months 1-3, \$150K)

- Partner with regional or mid-size insurer
- Test on 6 months of anonymized claims with known fraud outcomes
- Measure precision/recall against actual recovery cases
- Iterate on agent prompts and tool logic

2. Investigation quality scoring system (Months 2-4, \$100K)

- Build feedback loop: Did analyst agree with the dossier?
- Train a quality classifier to predict 'case strength' before analyst review
- Auto-triage: High-quality cases → priority queue, weak cases → deprioritized
- This makes the system learn what 'investigation ready' means

3. Production infrastructure (Months 4-6, \$250K)

- API for integration with existing fraud systems (FICO, SAS, etc.)
- Queue management and parallelization
- Audit logging and compliance dashboards
- Self-hosted LLM option for data residency requirements

What we're NOT building: Detection ML (commoditized), case management UIs (that's the payer's system), or provider audit workflow (compliance owns that).

We stay focused on **investigation automation** - the gap between alert and case file."

#15: "90 seconds is impressive - but is that with cached data or real-time API calls?"

ANSWER: "90 seconds is with:

- Cached dataset (50K claims pre-loaded)
- Real-time LLM API calls to Claude via Bedrock
- Real-time tool execution (anomaly detection, network analysis, etc.)

Full cold-start timing:

- Data generation (first time only): ~8-10 seconds
- Investigation reasoning: ~80-90 seconds (same as demo)
- **Total: ~100 seconds worst case**

The heavy part isn't data generation - it's the **8-12 sequential LLM calls** as agents reason through the case. Each call is ~5-10 seconds depending on complexity.

Cost per investigation: ~\$0.15 in Claude API calls (12K tokens input, 4K output avg).

We could optimize further with prompt caching (Claude supports this now) and parallel tool calls where possible, but 90 seconds felt like a good balance of thoroughness vs speed. Fraud analysts would be thrilled with 90-second preliminary investigations."

#16: "Your network analysis assumes shared patients = collusion. Isn't that normal in healthcare?"

ANSWER: "Absolutely normal - that's why we look at concentration, not just volume.

Legitimate referral example:

- Cardiologist refers 50 patients to cardiac surgeon
- But surgeon receives 500 referrals total from 30 different providers
- **Concentration: 10%** (normal)

Fraud ring example:

- 3 internal medicine providers refer 12 patients to pain clinic
- Pain clinic receives 15 total patients, 12 from these 3 providers
- **Concentration: 80%** (suspicious)

We also check:

- **Referral pattern shifts:** Did referrals suddenly spike?
- **Bi-directional patterns:** Are they referring to each other in a loop?
- **Shared patient outcomes:** Do shared patients have similar billing patterns (e.g., all getting the same high-value procedures)?
- **Geographic clustering:** Are they in the same building/network?

The tool is called `detect_fraud_ring` but it's really 'detect suspicious coordination patterns.' The agent still has to assess context - which is why the legitimate high-biller case gets dismissed despite high concentration due to specialty justification."

#17: "You claim 95% normal, 5% fraud - but real fraud rates are <1%. How does this affect your model?"

ANSWER: "You're right - this is overfit to minority class for demo purposes.

Real-world adjustment:

- At 0.5% fraud rate, Isolation Forest threshold would need recalibration
- We'd expect lower precision, same recall (more false positives to sift through)
- This is exactly where the **agent investigation layer becomes critical**

The agent workflow is actually **more valuable** at low fraud rates because you need more sophisticated filtering:

- Anomaly detector flags 1,000 claims (2% FP rate at 50K claims)
- 500 are fraud (0.5% × 100K claims assumed)
- 500 are false positives
- **Agents investigate all 1,000 and dismiss the 500 FPs**

At 5% fraud rate, simple rules might work. At 0.5% fraud rate, you need reasoning to separate signal from noise.

We used 5% for demo because:

1. Easier to find diverse fraud types in smaller dataset
2. Shows both fraud detection AND legitimate dismissal
3. Proves the agent logic works at higher signal

Fair criticism though - **production validation needs real fraud rate distribution**. That's the pilot test."

#18: "Your FAISS index searches billing rules - but rules change constantly. How do you keep this updated?"

ANSWER: "Currently: Manual rules.json update. Production: Automated ingestion pipeline needed.

Current state:

- ~150 billing rules manually encoded from CMS/OIG guidance
- FAISS vector index rebuilds on startup (~1 second)

Production version needs:

1. **RSS/API monitoring** of OIG fraud alerts, CMS transmittals
2. **NLP extraction** of new rule patterns from bulletins
3. **Version control** of rule changes (timestamp, source citation)
4. **Differential updates** to FAISS index (add new, deprecate old)
5. **Human review queue** for rule changes before deployment

This is very doable - it's an ETL problem, not a research problem. OIG publishes fraud alerts in structured format. CMS transmittals have consistent patterns.

Time estimate: 2-3 weeks to build automated ingestion + weekly update pipeline.

Good catch - this is a maintenance requirement we haven't built but would be essential for production. Rules drift would kill accuracy fast."

#19: "Isolation Forest gives you 0.92 anomaly score - but how did you calibrate that threshold?"

ANSWER: "We did basic ROC analysis on synthetic data with known ground truth.

Threshold testing:

- 0.75: High recall (95%) but lots of FPs (precision ~60%)
- 0.85: Balanced (recall 89%, precision 73%)
- 0.95: Very precise (95%) but misses fraud (recall ~65%)

We chose 0.85 as default because it catches most fraud while keeping agent workload manageable.

But here's the key: **The threshold isn't production-tuned - it's demo-tuned.**

In production, you'd:

1. Set threshold based on investigation capacity (how many cases can analysts handle?)
2. Monitor precision/recall over time as fraud patterns evolve
3. Use feedback loops (did this case lead to recovery?) to re-calibrate
4. Potentially use different thresholds by specialty or claim type

We proved the concept with reasonable threshold, but enterprise deployment would need **continuous threshold optimization** based on actual outcomes. This is ML Ops 101 - you can't set-and-forget anomaly thresholds."

#20: "This is cool - but is it a product or a demo?"

ANSWER: "Honest answer: 90% demo, 10% product.

What's production-ready:

- Agent workflow and reasoning logic (solid)
- Tool integration patterns (works reliably)
- Streaming UI and case presentation (polished enough)

What's not:

- Real data validation (synthetic only)
- Threshold calibration (demo-tuned, not production-tuned)
- Enterprise integration (no API for fraud system integration)
- Audit/compliance features (logging exists but not packaged)
- Self-hosted LLM option (cloud-only)
- Rules update pipeline (manual)

To get to production:

- 3 months with pilot customer
- \$100K in engineering (infrastructure, integration, testing)
- Compliance/legal review for healthcare deployment

But - the core innovation (multi-agent investigation workflow) is proven. The gap isn't 'does this work' - it's 'does this work at scale with real data and real compliance requirements.'

That's a typical hackathon-to-product gap. We're showing the **what** is possible. The **how to deploy** is the next phase."

#21: “You’re using Claude via Bedrock. What stops competitors from copying your prompts?”

ANSWER: "Nothing. And that's fine.

Our moat isn't the prompts - it's:

1. **Investigation workflow design** - The 3-agent split, tool allocation, rejection loop logic. This took iteration to get right.
2. **Healthcare-specific tools** - Anomaly detection calibrated for medical claims, network analysis for provider relationships, billing rules encoded from OIG/CMS. Domain knowledge embedded in tools.
3. **Integration & deployment** - Eventually: API connectors to fraud systems, compliance dashboards, audit trails, case management integration.
4. **Data flywheel** - With each deployment: analyst feedback → better prompts, threshold tuning, quality scoring. The system gets smarter with use.

Prompts are copyable. **Systems that work in production healthcare environments are not.**

Someone could copy our prompts and build their own version. Great! That validates the approach. But they'd still need to:

- Calibrate for real healthcare fraud patterns
- Integrate with payer systems
- Get compliance approval
- Build analyst trust

We have a 6-month head start on that learning curve. By the time someone copies the prompts, we've already iterated based on pilot feedback."

#22: “Who’s your target customer and are they actually willing to adopt AI for fraud detection?”

ANSWER: "Target: Regional and mid-size health insurers (500K-5M members).

Why them:

- Big enough to have fraud teams (5-20 analysts)
- Small enough to feel the pain (can't hire 50 analysts like UHG)
- Modern tech stacks (cloud-friendly, willing to try AI)
- Budget authority at VP level (faster sales cycle)

Willingness to adopt: Healthcare is slow on patient-facing AI (high risk, regulations). But **operational AI is moving fast:**

- Prior auth automation (already deployed at major payers)
- Claims routing and adjudication (widespread)
- Fraud scoring systems (FICO, SAS - incumbents)

The wedge: **We're not doing fraud detection (scary, high-stakes). We're doing investigation triage (helps humans, internal use, no patient impact).**

Sales angle: 'Your analysts are drowning in alerts. We give them a head start on every case.'

Expected sales cycle: 6-9 months (pilot → procurement → deployment). Not fast, but not medical device 5-year timelines either."



Quick Reference: Top 6 Critical Answers

1. **Synthetic Data Concern** → Real patterns from OIG cases + agent layer handles validation
2. **Demo Perfection Doubt** → Live LLM calls, cached data, can run different scenarios now
3. **Why 3 Agents** → Division of labor + rejection loop = better case quality
4. **ROI/Savings** → \$50M from prioritized investigation vs manual triage
5. **False Positive Rate** → 73% precision but agents dismiss the FPs
6. **Why Now** → Function-calling LLMs + LangGraph + commoditized fraud ML

Master these 6 and you can deflect or deep-dive on anything else.